OASIS VM 빌드 시스템 분석

개요

OASIS VM은 Yocto/OpenEmbedded 빌드 시스템을 사용하여 재현 가능한 빌드 환경을 제공합니다. Docker 컨테이너를 통해 일관된 빌드 환경을 보장하며, 여러 메타 레이어를 조합하여 최종 이미지를 생성합니다.

---

1. 빌드 아키텍처

전체 구조

```
Makefile → Docker 컨테이너 → Yocto 빌드 → 아티팩트 생성
```

주요 구성 요소

- **Makefile**: 빌드 오케스트레이션
- **Docker 환경**: 격리된 빌드 환경
- **Yocto 레이어**: 패키지 및 이미지 정의
- **BitBake**: 빌드 엔진

---

2. Makefile 기반 빌드 제어

주요 타겟

`make all` (기본 타겟)

```
all: build-outer
```

- 전체 빌드 프로세스 시작
- Git 서브모듈 준비 후 Docker 빌드 실행

`make prepare`

```
prepare:
@git submodule update --init --recursive
```

- Git 서브모듈 초기화 및 업데이트
- poky, meta-openembedded, meta-virtualization 등

`make build-outer`

```
build-outer: prepare
@$(DOCKER) run --rm -it \
-v $(shell pwd):/workdir \
-w /workdir \
$(BUILD_IMAGE)  \
make build-inner
```

- Docker 컨테이너에서 실제 빌드 실행
- 호스트 디렉토리를 `/workdir` 로 마운트

make build-inner

```
build-inner:
@source poky/oe-init-build-env build/ && \
bitbake oasis-vm-stage1 && \
bitbake oasis-vm-stage2-basic && \
bitbake oasis-vm-stage2-podman && \
bitbake ovmf
@rm -rf $(FINAL_ARTIFACTS_DIR)
@mkdir -p $(FINAL_ARTIFACTS_DIR)
@cp $(INTERMEDIATE_ARTIFACTS_DIR)/ovmf.tdx.fd $(FINAL_ARTIFACTS_DIR)
@cp $(INTERMEDIATE_ARTIFACTS_DIR)/oasis-vm-stage2-basic-tdx.tar.bz2 $(FINAL_ARTI
@cp $(INTERMEDIATE_ARTIFACTS_DIR)/oasis-vm-stage2-podman-tdx.tar.bz2 $(FINAL_ART
@cp $(INTERMEDIATE_ARTIFACTS_DIR)/bzImage-initramfs-tdx.bin $(FINAL_ARTIFACTS_DI
```

- Yocto 환경 초기화
- 모든 컴포넌트 빌드
- 최종 아티팩트 복사

빌드 환경 설정

```
# Docker 이미지
BUILD_IMAGE := crops/poky@sha256:fe6976cc917b6e1d6d11583ef628a16a165c91e76715608
```

```
# 디렉토리 설정

INTERMEDIATE_ARTIFACTS_DIR := build/tmp/deploy/images/tdx

FINAL_ARTIFACTS_DIR := build/artifacts
```

---

3. Yocto 레이어 구조

메타 레이어 계층

```
meta-oasis-vm/ # OASIS VM 특화 레이어

├── conf/ # 배포 및 머신 설정

├── recipes-containers/ # 컨테이너 관련

├── recipes-core/ # 핵심 이미지/패키지

├── recipes-devtools/ # 개발 도구

└── recipes-kernel/ # 커널 설정


meta-openembedded/ # 추가 패키지 레이어

meta-virtualization/ # 가상화 관련 레이어

meta-poky-patches/ # Poky 패치

poky/ # 기본 Yocto 레이어
```

레이어 설정 (meta-oasis-vm/conf/layer.conf)

```
BBPATH .= ":${LAYERDIR}"

BBFILES += "${LAYERDIR}/recipes-*/*/*.bb ${LAYERDIR}/recipes-*/*/*.bbappend"

BBFILE_COLLECTIONS += "meta-oasis-vm"

BBFILE_PATTERN_meta-oasis-vm = "^${LAYERDIR}/"

BBFILE_PRIORITY_meta-oasis-vm = "6"

LAYERVERSION_meta-oasis-vm = "1"

LAYERSERIES_COMPAT_meta-oasis-vm = "scarthgap"
```

---

4. 이미지 빌드 프로세스

Stage 1 이미지 빌드

레시피: `oasis-vm-stage1.bb`

```
DESCRIPTION = "Minimal image that includes the Oasis stage 1 init."

PACKAGE_INSTALL = "stage1-init ${VIRTUAL-RUNTIME_base-utils} ${ROOTFS_BOOTSTRA

INITRAMFS_MAXSIZE = "20000000" # 20MB 제한

IMAGE_FSTYPES = "${INITRAMFS_FSTYPES}"

inherit core-image
```

특별한 처리

```
# /dev/console 디바이스 생성
```

```
IMAGE_PREPROCESS_COMMAND:append = " stage1_prepare_rootfs"

stage1_prepare_rootfs() {

mkdir -p ${IMAGE_ROOTFS}/dev

mknod -m 622 ${IMAGE_ROOTFS}/dev/console c 5 1

}
```

패키지 구성

- **stage1-init**: 초기화 스크립트
- **busybox**: 기본 유틸리티
- **cryptsetup**: 암호화 볼륨 관리
- **e2fsprogs-mke2fs**: 파일시스템 도구
- **util-linux-blockdev**: 블록 디바이스 도구

Stage 2 이미지 빌드

공통 설정 (common.inc)

```
OASIS_BASIC_INSTALL = "stage2-base busybox libgcc libssl ca-certificates"

IMAGE_FEATURES = ""

IMAGE_FSTYPES = "tar.bz2" # 압축 아카이브 형태

inherit core-image

IMAGE_ROOTFS_SIZE = "8192"
```

기본 버전 ([oasis-vm-stage2-basic.bb](#))

```
PACKAGE_INSTALL = "${OASIS_BASIC_INSTALL} ${VIRTUAL-RUNTIME_base-utils} ${ROOT
export IMAGE_BASENAME = "oasis-vm-stage2-basic"
```

Podman 버전 (oasis-vm-stage2-podman.bb)

```
OASIS_PODMAN_INSTALL = "stage2-podman"
PACKAGE_INSTALL = "${OASIS_BASIC_INSTALL} ${OASIS_PODMAN_INSTALL} ${VIRTUAL-RU
export IMAGE_BASENAME = "oasis-vm-stage2-podman"
```

---

5. 커널 빌드 설정

커널 레시피 확장 (linux-yocto%.bbappend)

```
KMACHINE:tdx ?= "common-pc-64"
COMPATIBLE_MACHINE:tdx = "tdx"

KERNEL_FEATURES:append:tdx=" features/scsi/disk.scc"
KERNEL_FEATURES:append:tdx=" cfg/virtio.scc cfg/paravirt_kvm.scc cfg/fs/ext4.scc
KERNEL_FEATURES:append:tdx=" tdx.scc security-mitigations.scc disk-encryption.sc
```

커널 기능 모듈

- **tdx.scc**: TDX 지원
- **security-mitigations.scc**: 보안 완화 기법
- **disk-encryption.scc**: 디스크 암호화 지원
- **podman.scc**: 컨테이너 런타임 지원

추가 설정 파일

```
# OASIS VM 특화 설정
SRC_URI += "file://oasisvm.scc file://oasisvm.cfg"

# 디스크 암호화 설정
SRC_URI += "file://disk-encryption.scc file://disk-encryption.cfg"
```

---

6. OVMF 펌웨어 빌드

빌드 설정

```
SUMMARY = "OVMF - UEFI firmware for Qemu and KVM"
SRC_URI = "gitsm://github.com/tianocore/edk2.git;branch=master;protocol=https"
PV = "edk2-stable202408"
SRCREV = "b158dad150bf02879668f72ce306445250838201"
```

패키지 설정

```
PACKAGECONFIG += "${@bb.utils.contains('MACHINE_FEATURES', 'tdx', 'tdx', '', d
PACKAGECONFIG[tdx] = ",,,"
```

TDX 빌드 프로세스

```
do_compile:class-target() {
# 일반 OVMF 빌드
${S}/OvmfPkg/build.sh $PARALLEL_JOBS -a $OVMF_ARCH -b RELEASE -t ${FIXED_G
# TDX 지원 빌드
if ${@bb.utils.contains('PACKAGECONFIG', 'tdx', 'true', 'false', d)}; then
build_dir="${S}/Build/IntelTdx/RELEASE_${FIXED_GCCVER}"
rm -rf ${S}/Build/IntelTdx
${S}/OvmfPkg/build.sh $PARALLEL_JOBS -a $OVMF_ARCH -b RELEASE -t ${FIXED_G
ln ${build_dir}/FV/OVMF.fd ${WORKDIR}/ovmf/ovmf.tdx.fd
fi
}
```

---

7. 패키지 관리

Stage 2 Base 패키지 (stage2-base.bb)

```
RDEPENDS:${PN} = " \
```

```
busybox \
busybox-udhcpc \
cryptsetup \
e2fsprogs-mke2fs \
e2fsprogs-resize2fs \
e2fsprogs-e2fsck \
"
```

파일시스템 구조 생성

```
do_install() {
ln  -s  lib ${D}/lib64
install  -d ${D}/proc
install  -d ${D}/sys
install  -d ${D}/dev
install  -d ${D}/tmp
install  -d ${D}/run
install  -d ${D}/storage
install  -d ${D}${sysconfdir}/oasis
install  -d ${D}${sysconfdir}/oasis/post-registration.d
install  -m  0755 ${WORKDIR}/init.post-registration ${D}${sysconfdir}/oasis/init
install  -m  0755 ${WORKDIR}/00-network ${D}${sysconfdir}/oasis/post-registratio
# tmpfs 심볼릭 링크
```

```
ln -s /tmp/resolv.conf ${D}${sysconfdir}/resolv.conf

}
```

Stage 2 Podman 패키지 (stage2-podman.bb)

```
RDEPENDS:${PN} = " \
stage2-base \
podman \
podman-compose \
python3-unixadmin \
python3-json \
python3-logging \
"
```

---

8. 컨테이너 지원

Podman 설정

```
# meta-oasis-vm/recipes-containers/podman/
├── podman/
└── podman-compose/
```

컨테이너 호스트 설정

```
# meta-oasis-vm/recipes-containers/container-host-config/
```

런타임 의존성

- **podman**: 컨테이너 런타임
- **podman-compose**: 컨테이너 오케스트레이션
- **python3-unixadmin**: 시스템 관리 도구

---

9. 빌드 최적화

재현 가능한 빌드

```
# 디버그 프리픽스 맵핑
export GCC_PREFIX_MAP = "${DEBUG_PREFIX_MAP} -Wno-stringop-overflow -Wno-maybe-
export NASM_PREFIX_MAP = "--debug-prefix-map=${WORKDIR}=${TARGET_DBGSRC_DIR}"
```

병렬 빌드

```
PARALLEL_MAKE = "" # OVMF는 병렬 빌드 비활성화
PARALLEL_JOBS="${@oe.utils.parallel_make_argument(d, '-n %d')}"
```

크기 최적화

```
# Stage 1 크기 제한
INITRAMFS_MAXSIZE = "20000000" # 20MB

# Stage 2 크기 설정
IMAGE_ROOTFS_SIZE = "8192"
IMAGE_ROOTFS_EXTRA_SPACE = "0"
```

---

10. 개발 및 디버깅

개발 환경 진입

```
make shell
```

- Docker 컨테이너에서 bash 셸 실행
- 수동 빌드 및 디버깅 가능

개별 컴포넌트 빌드

```
source poky/oe-init-build-env build/
bitbake oasis-vm-stage1 -c compile
bitbake oasis-vm-stage1 -c install
```

빌드 정리

```
make  clean  # build/tmp 디렉토리 삭제
```

로그 및 디버깅

- **빌드 로그**: `build/tmp/work/`
- **이미지 로그**: `build/tmp/deploy/images/tdx/`
- **패키지 로그**: `build/tmp/work/tdx-poky-linux/`

---

11. 출력 아티팩트

최종 빌드 결과

```
build/artifacts/
├── ovmf.tdx.fd # TDX 지원 UEFI 펌웨어
├── stage1.bin # Stage 1 커널 + initramfs
├── stage2-basic.tar.bz2 # 기본 Stage 2 루트 파일시스템
└── stage2-podman.tar.bz2 # Podman 지원 Stage 2 루트 파일시스템
```

중간 빌드 결과

```
build/tmp/deploy/images/tdx/
├── bzImage-initramfs-tdx.bin # Stage 1 커널
├── oasis-vm-stage2-basic-tdx.tar.bz2 # Stage 2 기본
├── oasis-vm-stage2-podman-tdx.tar.bz2 # Stage 2 Podman
```

```
└── ovmf.tdx.fd # OVMF 펌웨어
```

---

12. 확장성 및 커스터마이징

새로운 Stage 2 버전 추가

1. `meta-oasis-vm/recipes-core/images/` 에 새 레시피 생성
2. `common.inc` 상속
3. 필요한 패키지 추가
4. Makefile에 빌드 타겟 추가

커널 기능 추가

1. `meta-oasis-vm/recipes-kernel/linux/linux-yocto/` 에 설정 파일 추가
2. `.scc` 및 `.cfg` 파일 생성
3. `linux-yocto%.bbappend` 에 기능 추가

새로운 패키지 추가

1. `meta-oasis-vm/recipes-*/` 에 적절한 디렉토리 생성
2. `.bb` 레시피 파일 작성
3. 의존성 및 설치 규칙 정의

이 빌드 시스템은 모듈화되고 확장 가능한 구조로 설계되어 있어, 새로운 기능 추가나 기존 기능 수정이 용이합니다.